

Product Backlog – *Pastry Panic*

Team: Davut Mert Ahmetoglu & Bera Uzun

Topics of Choice:

- 1. User Experience (UX) in Game Design**
- 2. Event Handling in Java Swing**

1. Core Game Loop

How to demo: Launch the game. The player can move, items fall, and collisions are detected.

Notes: Maintain stable FPS. Game supports pause and restart.

2. Character System (Mert & Bera)

How to demo: Two characters, Mert Ahmetoglu and Bera Uzun, can be selected or alternate turns.

Notes: Each has a unique sprite. Controls and movement logic remain identical.

3. Falling Items Logic

How to demo: Pastries and rotten apples drop from the top of the screen and disappear upon collision or hitting the ground.

Notes: Use ItemType enum (Pastry, Rotten Object). Randomize spawn position and speed.

4. Collision Detection & Health/Energy System

How to demo: Collecting a pastry increases energy; touching a rotten apple decreases health. UI bars update in real-time.

Notes: Collision handled via Rectangle.intersects(). Proper game-state updates required.

5. Background Music & Sound Effects

How to demo: Background music starts automatically. Collision triggers a sound effect.

Notes: Use SoundManager. Include a toggle for sound on/off (UX element).

6. Game Over & Restart Flow

How to demo: When health reaches zero, a “Game Over” screen appears. Clicking “Restart” restarts the game.

Notes: Swing ActionListener manages transitions and button actions.

7. UX – Visual Feedback

How to demo: When energy increases, the character briefly glows or shows a short animation.

Notes: Reinforces reward feedback through animation and sound cues.

8. UX – Difficulty Progression

How to demo: As time passes, items fall faster, increasing challenge gradually.

Notes: Timer scales difficulty linearly or exponentially to maintain player engagement.

9. UX – Main Menu & Instructions

How to demo: A main menu appears on launch with “Start Game,” “How to Play,” and “Exit” buttons.

Notes: Clean layout with large buttons and soft background colors for accessibility.

10. Event Handling – Keyboard Input

How to demo: Player moves left and right with arrow keys. Holding down keys ensures smooth movement.

Notes: Implement using Swing Key Bindings (instead of KeyListener) for better responsiveness and thread safety.

11. Event Handling – Mouse Input

How to demo: Player can navigate menus and click buttons to start or exit the game.

Notes: Use ActionListener and MouseListener for menu interaction.

12. Timer & Game State Events

How to demo: Game updates every 50ms using a javax.swing.Timer. Timer pauses when the game is paused.

Notes: Central event loop for animation, collision, and difficulty progression.

13. Score System & UI Overlay

How to demo: The player's score and survival time are displayed at the top of the screen. Score increases with time.

Notes: Minimalist design. Prioritize clarity and visibility for UX consistency.

14. Assets Management

How to demo: All sprites (characters, background, items) load correctly and are scaled to resolution.

Notes: Managed in Assets.java. Ensure no lag or missing textures.

15. Final UX Polish

How to demo: Colors, typography, and layout are visually consistent. All UI elements readable and aligned.

Notes: Apply UX principles: clarity, consistency, minimalism, and visual hierarchy.

16. Testing & Debug Tools

How to demo: Enable debug mode with slower item fall speed and console output for collisions and states.

Notes: Used for verifying logic and performance before final build.

Priority Overview

Priority	Feature	Category
1.	Core Loop, Character System, Collision	Gameplay
2.	Sound, Timer, Keyboard Input	Event Handling
3.	Menu, Feedback, Difficulty, Score	UX
4.	Polish, Debug, Testing	Final Touches